COVID-19 Detection from Chest X-Ray Images

A Code-Based Project Report

A technical report prepared from the supplied Python notebook *Final_code.ipynb*. The report documents the implementation, model training, test-set evaluation, and explainability workflow represented in the code.

Project area	Medical image classification / deep learning
Classes	COVID and NORMAL
Input resolution	224 × 224 pixels
Models evaluated	Custom CNN, VGG16, ResNet50, DenseNet121, InceptionV3, Xception
Evaluation split	2,171 training; 464 validation; 465 test images

Prepared from the implementation and recorded notebook outputs.

Abstract

This project implements a binary image-classification pipeline to distinguish chest X-ray images labelled COVID from those labelled NORMAL. The notebook uses OpenCV for grayscale image preprocessing, TensorFlow/Keras for model construction and training, and scikit-learn for data splitting and evaluation. Six architectures are compared: a custom convolutional neural network (CNN) and five ImageNet-pretrained convolutional backbones—VGG16, ResNet50, DenseNet121, InceptionV3, and Xception. The recorded test classification reports show that VGG16 achieved the highest test accuracy (97%), followed by ResNet50 (96%), DenseNet121 (94%), Custom CNN (92%), InceptionV3 (85%), and Xception (84%). Grad-CAM code is also included to visualize image regions that influence a selected model's prediction. These are experimental results from this notebook and are not evidence of clinical readiness.

1. Introduction

Chest X-ray images contain visual patterns that can be investigated with computer-vision methods. This project explores whether deep-learning models can classify images into two dataset labels: COVID and NORMAL. The notebook is structured as an experimental comparison rather than a clinically validated diagnostic product. Its main purpose is to build a repeatable preprocessing and training workflow, compare different neural-network architectures, and inspect model predictions with standard classification metrics and Grad-CAM visualizations.

2. Objectives

- Prepare chest X-ray images in a consistent 224 × 224 pixel format.
- Apply image enhancement and denoising operations before model training.
- Train one custom CNN and compare it with five pretrained CNN architectures.
- Evaluate models on a held-out test set using accuracy, precision, recall, F1-score, and confusion matrices.
- Use Grad-CAM to produce a visual explanation for a selected prediction.
- Document practical limitations that should be addressed before any real-world medical use.

3. Dataset and Preprocessing

3.1 Dataset organization

The code expects a Google Drive directory at `/content/drive/MyDrive/Defence` with two subfolders named `COVID` and `NORMAL`. Each readable image is assigned a numeric class label based on the category list. The notebook silently skips files that raise an exception during preprocessing. Because the dataset itself is not included with the notebook, the source, consent status, patient-level separation, and exact number of unique patients cannot be independently verified from the code alone.

3.2 Image preprocessing

- Read each image as grayscale using OpenCV.
- Resize the image to 224 × 224 pixels.
- Scale pixel values by dividing by 255, then apply histogram equalization.
- Apply CLAHE (contrast-limited adaptive histogram equalization) to improve local contrast.
- Apply a 3 × 3 Gaussian blur for denoising.
- Expand the image channel dimension to produce a single-channel input for the custom CNN.

For pretrained models, the grayscale image is repeated across three channels so it can be supplied to backbones expecting RGB-shaped input. The notebook does not show a separate model-specific preprocessing function such

as the application-specific preprocessing utilities commonly used with pretrained ImageNet models; this is a consideration when interpreting the comparison.

3.3 Train, validation, and test split

The notebook first reserves 15% of the data for testing, then splits the remaining data into training and validation sets using stratification and a fixed random seed of 42. The recorded split sizes are 2,171 training images, 464 validation images, and 465 test images. Stratification aims to preserve class proportions across splits. The code does not demonstrate patient-level grouping, so if multiple images from the same patient exist, potential data leakage should be checked.

4. Model Design and Training

4.1 Custom CNN

The custom network accepts a 224 × 224 × 1 image. It contains two convolutional layers (32 and 64 filters) with ReLU activations, each followed by max pooling, then a flattening layer, a 128-unit dense layer, dropout of 0.3, and a two-unit softmax output layer. The model is compiled with the Adam optimizer and categorical cross-entropy loss.

4.2 Transfer-learning models

The notebook loads the convolutional feature extractors VGG16, ResNet50, DenseNet121, InceptionV3, and Xception with ImageNet weights and without their original classification heads. The pretrained base is frozen. A global-average-pooling layer, a 128-unit ReLU dense layer, dropout of 0.3, and a two-class softmax layer are added. These models are compiled with Adam and categorical cross-entropy. Since the backbone layers are frozen, this implementation primarily trains the newly added classification head rather than fine-tuning the full pretrained network.

4.3 Training procedure

Each model is trained for five epochs in the recorded notebook run, using batches of 32 images and the validation split. Training histories include accuracy and loss values. The notebook saves trained models to the configured Google Drive output directory. Runtime availability of a GPU depends on the active Colab session and is not guaranteed by the code itself.

5. Evaluation Method

The code evaluates each model on the same 465-image test set. It produces predicted class probabilities, selects the class with the largest probability, draws a confusion matrix, and prints a scikit-learn classification report. The report includes per-class precision, recall, F1-score, support, overall accuracy, macro average, and weighted average. The notebook also creates a bar chart comparing test accuracy and plots training/validation accuracy and loss curves.

Precision measures how often predictions for a class are correct; recall measures how many actual examples of that class are identified; and F1-score combines precision and recall. In a medical screening context, false negatives and false positives have different consequences, so accuracy alone is not sufficient.

6. Recorded Results

The following values are transcribed from the saved classification-report outputs in the supplied notebook. Percentages are rounded to whole numbers where shown; class-level figures are rounded to two decimal places.

Model	Test accuracy	COVID precision	COVID recall	COVID F1	NORMAL F1
Custom CNN	92%	0.90	0.94	0.92	0.92

Model	Test accuracy	COVID precision	COVID recall	COVID F1	NORMAL F1
VGG16	97%	0.96	0.99	0.97	0.97
ResNet50	96%	0.93	1.00	0.96	0.96
DenseNet121	94%	0.96	0.92	0.94	0.94
InceptionV3	85%	0.83	0.87	0.85	0.84
Xception	84%	0.78	0.95	0.86	0.82

Interpretation: VGG16 recorded the strongest overall test result in this run, with 97% accuracy and 0.97 F1-score for both classes. ResNet50 followed at 96% accuracy; its COVID recall was 1.00 in the recorded report, while NORMAL recall was 0.92. The Custom CNN reached 92% accuracy. InceptionV3 and Xception produced lower overall accuracy in this run. Results are specific to this dataset split, preprocessing pipeline, initialization, and five-epoch training setup; they should not be generalized to other hospitals or populations without further testing.

Note: the results above are taken from the notebook's printed test classification reports, rather than estimated from the separate narrative report. The code's measured test accuracies differ from some values stated in the previously supplied PDF, so this report prioritizes the notebook outputs.

7. Grad-CAM Explainability

The notebook includes a Grad-CAM implementation for ResNet50. It calculates gradients of a selected class score with respect to a convolutional feature map, combines the feature maps into a heatmap, and overlays that heatmap on the input X-ray. The configured ResNet50 layer name is `conv5_block3_out`. The intended output is a qualitative visualization of image regions associated with the model's prediction.

Grad-CAM is not a proof that the model is reasoning medically correctly. Heatmaps should be checked for artifacts, image borders, labels, or other shortcuts, and should not be treated as a substitute for clinical interpretation.

8. Discussion

- VGG16 achieved the highest recorded test accuracy among the six models in the saved run.
- ResNet50 performed closely behind VGG16 and had the highest recorded COVID recall in the classification report.
- The custom CNN provides a comparatively simple baseline and achieved 92% test accuracy in the saved run.
- InceptionV3 and Xception performed less strongly under the specific training configuration used in this notebook.
- Because the pretrained backbones are frozen and training lasts five epochs, additional experiments could test fine-tuning, learning-rate schedules, early stopping, and class-specific operating thresholds.
- A rigorous comparison should also verify consistent preprocessing, repeat experiments across random seeds, and report confidence intervals.

9. Limitations and Ethical Considerations

- The notebook alone does not verify dataset provenance, licensing, patient consent, demographic representation, or label quality.
- The code does not establish patient-level separation between training, validation, and test data.
- A single held-out test set and one recorded run are insufficient to establish performance across clinical settings.

- Accuracy and classification reports do not demonstrate clinical utility, calibration, or robustness to changes in scanner, hospital, image quality, or disease prevalence.
- The implementation is a research prototype. It must not be used to diagnose, rule out, or guide treatment for COVID-19. Any medical application requires expert oversight, independent external validation, privacy safeguards, and appropriate regulatory review.

10. Conclusion

The supplied notebook implements a full experimental pipeline for binary chest X-ray classification, from preprocessing and data splitting through model training, comparative evaluation, and Grad-CAM visualization. In the recorded run, VGG16 achieved 97% test accuracy, ResNet50 96%, DenseNet121 94%, Custom CNN 92%, InceptionV3 85%, and Xception 84%. These results support further controlled research but do not establish readiness for clinical deployment. The most important next steps are to validate dataset integrity and patient-level splits, standardize pretrained-model preprocessing, run repeated experiments, assess external generalization, and involve qualified medical professionals in any future evaluation.

Appendix A. Main Software Libraries

The code uses Python, TensorFlow/Keras, NumPy, OpenCV (cv2), scikit-learn, Matplotlib, Seaborn, and tqdm. The notebook is designed around Google Colab and Google Drive paths.

Appendix B. Reproducibility Notes

- Mount Google Drive in Colab and ensure the dataset path contains the COVID and NORMAL folders.
- Check the class-to-index mapping before interpreting confusion matrices or per-class metrics.
- Confirm that saved models and evaluation outputs correspond to the same notebook version.
- Record library versions, hardware, random seeds, and dataset version for future experiments.
- Review the duplicate Grad-CAM helper definitions in the notebook and retain one tested implementation in a cleaned version.